外れ値に強い回帰

損失を変える・データを選ぶ

rkskmt

ここまでの道具

- **線形回帰**： **MSE を勾配降下法で小さくして**、体長から重さを予測する直線を fit した
- **MSE**：予測と実際のズレを二乗して平均した損失

> **ⓘ 今回の主題**
>
> - データに間違った記録が混ざっていたら、fit した直線はどうなるか
> - 間違った記録に引っ張られない fit の方法を 2 つ作る

準備：魚の体長と重さ

- 線形回帰で使った 80 匹の体長と重さを使う

▸ 魚データの定義（クリックして展開）

伝票に打ち間違いが2枚あった

- 魚市場の伝票（1匹につき1枚、体長と重さ）80 枚のうち 2 枚で、重さの欄に 0 が1つ多く打たれていた
- 間違っていたのは、いちばん大きい 2 匹（2,457 g → 24,570 g、2,474 g → 24,740 g）
- この 80 枚で直線を fit し直す

Code

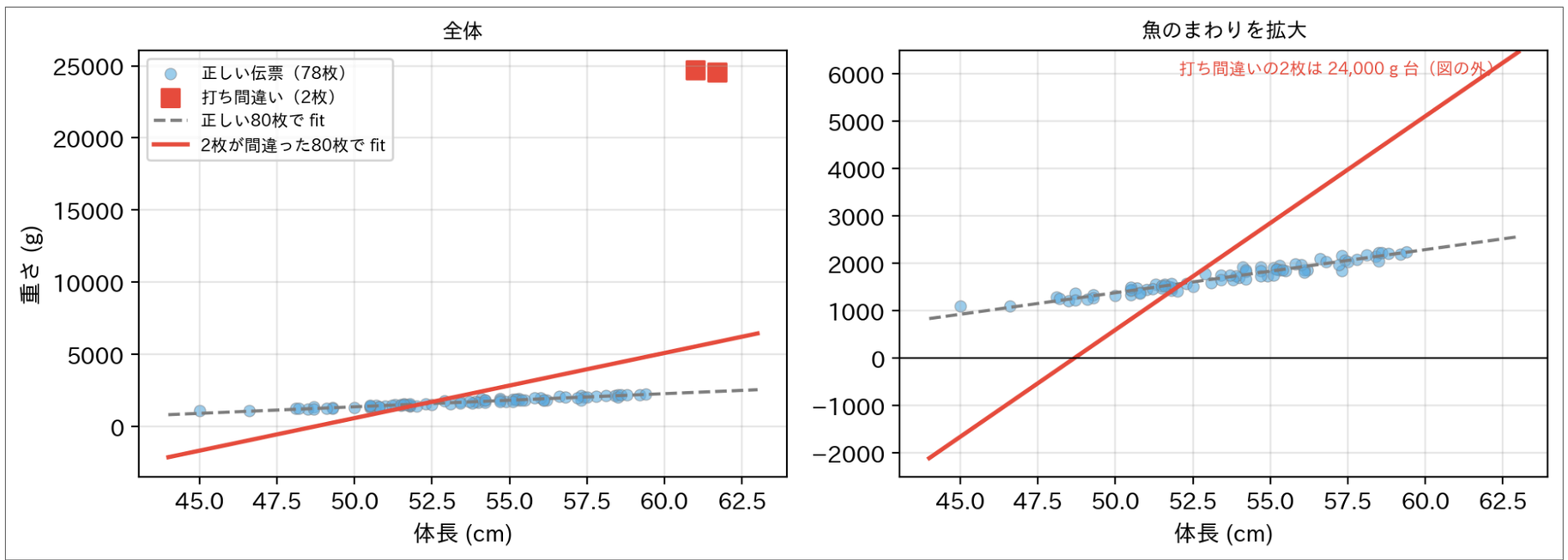
```
from sklearn.linear_model import LinearRegression

bad = [3, 6]                                          # 打ち間違えた2枚（いちばん大きい2匹）
weights_typo = weights.copy()
weights_typo[bad] *= 10                               # 0 が1つ多い ＝ 10倍

reg_ok   = LinearRegression().fit(lengths.reshape(-1, 1), weights)        # 正しい80枚
reg_typo = LinearRegression().fit(lengths.reshape(-1, 1), weights_typo)   # 2枚が間違った80枚
print(f"正しい80枚で fit  : 体長 1cm あたり {reg_ok.coef_[0]:.1f} g")
print(f"2枚が間違った80枚 : 体長 1cm あたり {reg_typo.coef_[0]:.1f} g")
```

Result

```
正しい80枚で fit  : 体長 1cm あたり 91.0 g
2枚が間違った80枚 : 体長 1cm あたり 451.2 g
```



ⓘ 結果

- たった 2 枚で、傾きは 91 g/cm から 451 g/cm になった
- 直線は魚の群れから離れた
  - → 45 cm の魚の予測は −1,660 g（マイナスの重さ）
  - → 62 cm の魚の予測は 6,010 g（実際は 2,400 g 台）
- 間違った 2 枚が、正しい 78 枚の予測を全部壊している

二乗で測ると、2枚のズレが78枚を圧倒する

- MSE は「ズレの二乗」の平均なので、ズレが 100 倍なら損失への寄与は 10,000 倍
- 正しい 80 枚の直線からの、各伝票のズレを比べる

Code

```
pred_ok = reg_ok.predict(lengths.reshape(-1, 1))          # 正しい80枚の直線による予測
error   = weights_typo - pred_ok                          # 各伝票のズレ（g）
ok      = np.ones(80, dtype=bool)
ok[bad] = False                                           # 正しい78枚だけ True
print(f"正しい78枚のズレ   : 最大 {np.abs(error[ok]).max():.0f} g / 二乗の合計 {np.sum(error[ok]**2):,.0f}")
print(f"打ち間違い2枚のズレ: 最小 {np.abs(error[bad]).min():.0f} g / 二乗の合計 {np.sum(error[bad]**2):,.0f}")
```

Result

```
正しい78枚のズレ   : 最大 206 g / 二乗の合計 432,045
打ち間違い2枚のズレ: 最小 22125 g / 二乗の合計 989,425,380
```

要するに

- 2 枚の二乗和は 78 枚の 2,290 倍
- MSE を下げるには、正しい 78 枚を全部見捨ててでも、この 2 枚に直線を寄せたほうが「得」になる

外れ値（outlier）

- 他の記録から大きく離れた記録（ここでは打ち間違いの 2 枚）
- 最小二乗法は二乗で測るので、少数の外れ値に直線が引っ張られる

MAE

対策：ズレを二乗せずに測る（MAE）

- 外れ値に引っ張られる原因は二乗なので、二乗をやめる
- ズレの**絶対値**を平均する

$$L(a,b) = \frac{1}{N}\sum_{i}\left|\underbrace{ax_i + b}_{\text{予測}} - \underbrace{t_i}_{\text{実際}}\right|$$

- **MAE（Mean Absolute Error / 平均絶対誤差）**と呼ぶ
- **MSE** との違いは、二乗が絶対値に変わったことだけ
- 正しい 80 枚の直線からのズレを、絶対値で比べる

Code

```
print(f"正しい78枚のズレ: 絶対値の合計 {np.sum(np.abs(error[ok])):,.0f} g")
print(f"打ち間違い2枚のズレ: 絶対値の合計 {np.sum(np.abs(error[bad])):,.0f} g")
```

Result

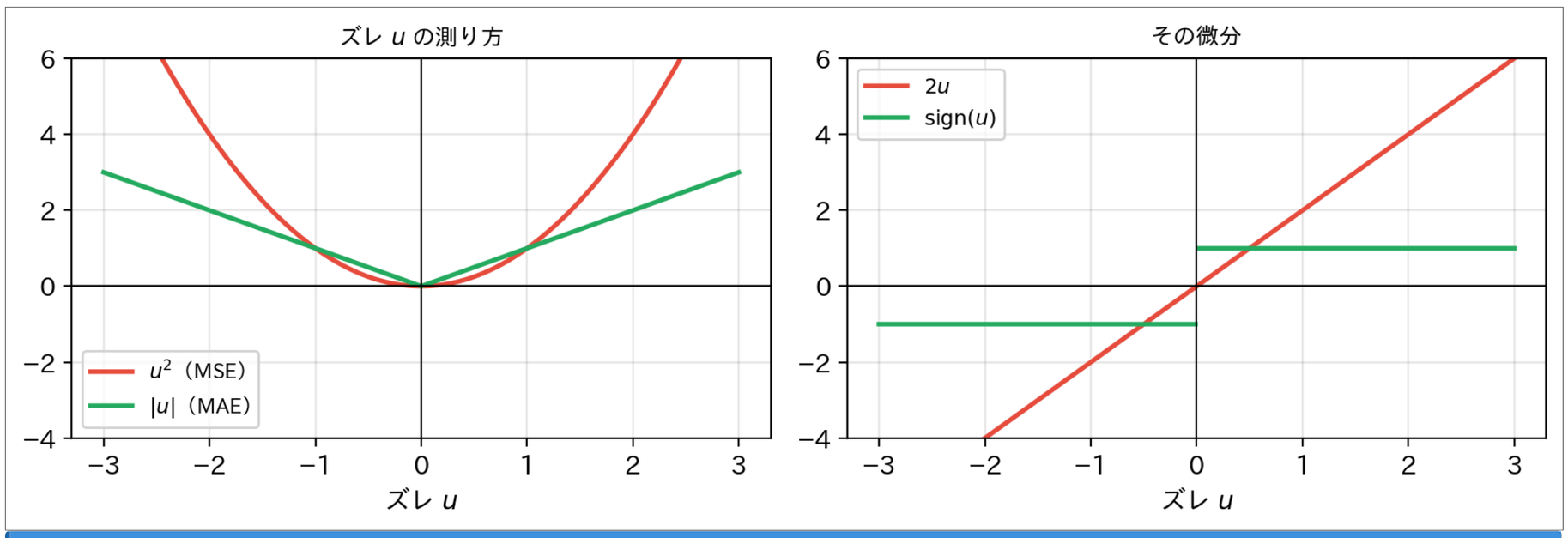
```
正しい78枚のズレ: 絶対値の合計 4,734 g
打ち間違い2枚のズレ: 絶対値の合計 44,484 g
```

ⓘ 結果

- 2 枚の絶対値の和は、78 枚の 9.4 倍
  → 二乗の和では **2,290 倍** だった
- ズレが 100 倍でも、損失への寄与は 100 倍で済む

絶対値の微分は、符号だけになる

- 勾配降下法には、損失の微分が要る
- u^2 の微分は $2u$、$|u|$ の微分は u の符号



ⓘ 符号を返す関数 sign

- $\mathrm{sign}(u)$：$u > 0$ なら $+1$、$u < 0$ なら -1、$u = 0$ なら 0
  → コードでは **`np.sign`**
- $|u|$ は $u = 0$ で折れていて微分できない
  → **`np.sign(0)`** は 0 を返すので、勾配降下法はそのまま続けられる

MAE の勾配：ズレの大きさが消える

L を a と b でそれぞれ微分する：

$$\frac{\partial L}{\partial a} = \frac{1}{N}\sum_i \operatorname{sign}(ax_i + b - t_i)\, x_i \qquad \frac{\partial L}{\partial b} = \frac{1}{N}\sum_i \operatorname{sign}(ax_i + b - t_i)$$

- **MSE の勾配** の「ズレ」が、「ズレの符号」に変わる

損失	1 枚の伝票が勾配に渡すもの	22,000 g ずれた伝票
MSE	ズレそのもの	100 g ずれた伝票の 220 倍の力で直線を引く
MAE	ズレの符号（$+1$ か -1）	100 g ずれた伝票と同じ力で直線を引く

MAE で fit する

- **線形回帰の勾配降下法** の、勾配の 2 行を書き換える
- 学習率 0.1 と更新 100 回は変えない

Code

```
x = (lengths - lengths.mean()) / lengths.std()                    # 標準化した体長
t = (weights_typo - weights_typo.mean()) / weights_typo.std()     # 標準化した重さ（打ち間違い入り）

def to_gram(a, b, cm):
    """標準化したデータで fit した直線 (a, b) で、体長 cm の魚の重さ（g）を予測する"""
    x_cm = (cm - lengths.mean()) / lengths.std()
    return (a * x_cm + b) * weights_typo.std() + weights_typo.mean()

a, b = 0.0, 0.0
lr = 0.1
for i in range(1, 101):
    pred   = a * x + b
    grad_a = np.mean(np.sign(pred - t) * x)     # MSE では 2 * np.mean((pred - t) * x)
    grad_b = np.mean(np.sign(pred - t))         # MSE では 2 * np.mean(pred - t)
    a -= lr * grad_a
    b -= lr * grad_b
    if i in (1, 3, 10, 50, 100):
        print(f"更新 {i:3d}回 | 45 cm の予測 {to_gram(a, b, 45):6,.0f} g | 62 cm の予測 {to_gram(a, b, 62):6,.0f
```

Result

```
  更新   1回 | 45 cm の予測  1,830 g | 62 cm の予測  2,029 g
  更新   3回 | 45 cm の予測    539 g | 62 cm の予測  2,712 g
  更新  10回 | 45 cm の予測  1,110 g | 62 cm の予測  2,537 g
  更新  50回 | 45 cm の予測  1,066 g | 62 cm の予測  2,737 g
  更新 100回 | 45 cm の予測  1,094 g | 62 cm の予測  2,711 g
```

ⓘ 結果

- 100 回更新後の予測は、45 cm で 1,094 g、62 cm で 2,711 g
  → MSE では **−1,660 g と 6,010 g** だった
- ただし 50 回と 100 回で値が違い、直線はまだ動いている

MAE の勾配降下法は、止まらずに揺れ続ける

- 更新を 300 回に増やす
- 更新のたびに、45 cm の魚の予測を記録する

Code

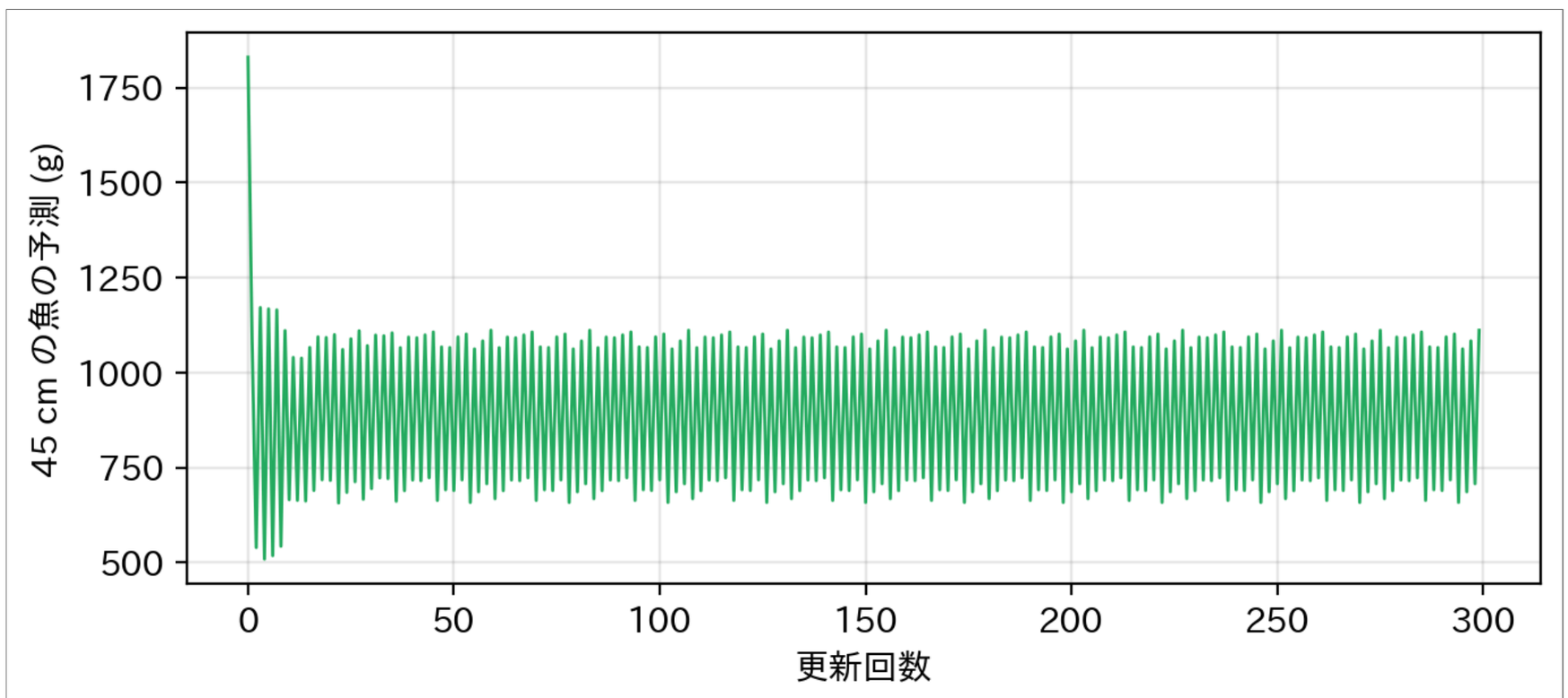
```
def fit_mae(lr, n_updates):
    """MAE の勾配降下法で fit し、更新ごとの「45 cm の魚の予測（g)」も返す"""
    a, b = 0.0, 0.0
    history = []
    for i in range(n_updates):
        pred = a * x + b
        a -= lr * np.mean(np.sign(pred - t) * x)
        b -= lr * np.mean(np.sign(pred - t))
        history.append(to_gram(a, b, 45))
    return a, b, history

a, b, history = fit_mae(lr=0.1, n_updates=300)
print(f"最後の50回の予測: {min(history[-50:]):,.0f} g から {max(history[-50:]):,.0f} g")

plt.figure(figsize=(7, 3.2))
plt.plot(history, color="#27ae60", lw=1)
plt.xlabel("更新回数"); plt.ylabel("45 cm の魚の予測 (g)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Result

```
最後の50回の予測: 658 g から 1,111 g
```



ⓘ 結果

- 300 回更新しても、予測は 658 g と 1,111 g のあいだを行き来する
- MSE のときは、 **50 回あたりで動かなくなった**

揺れる理由：MAE の 1 歩は、谷底の近くでも縮まない

損失	勾配の大きさを決めるもの	谷底に近づくと
MSE	ズレの大きさ	勾配が 0 に近づき、1 歩が自然に縮む
MAE	伝票が直線の上にあるか下にあるか	谷底まで 1 g でも 1,000 g でも、1 歩は同じ

- 学習率 0.1 の 1 歩を、重さ（g）に直す
  → 直線の高さ b の 1 歩は「学習率 × 勾配」で、MAE の勾配は最大 1
  → 標準化した重さの 1 は、標準偏差 1 つ分の重さ

Code

```
print(f"重さの標準偏差: {weights_typo.std():,.0f} g")
print(f"学習率 0.1 の 1 歩: 最大 {0.1 * weights_typo.std():,.0f} g")
print(f"正しい伝票のズレ: 最大 {np.abs(error[ok]).max():,.0f} g")
```

Result

```
重さの標準偏差: 3,596 g
学習率 0.1 の 1 歩: 最大 360 g
正しい伝票のズレ: 最大 206 g
```

ⓘ 結果

- 1 歩が最大 360 g で、正しい伝票のズレ（最大 206 g）より大きい
- 1 歩で伝票の群れをまたいでしまい、次の 1 歩で戻る

Tweak 学習率を小さくして揺れを抑える



- さっきのコード の `fit_mae(lr=0.1, n_updates=300)` を書き換えて実行

1	学習率を 1/10 にせよ	`lr=0.1` → `lr=0.01`
2	さらに 1/10 にせよ	`lr=0.01` → `lr=0.001`
3	更新回数を 10 倍にせよ	`n_updates=300` → `n_updates=3000`（`lr=0.001` のまま）

チェックポイント

- **`lr=0.01`**：最後の 50 回の予測は 873 g から 876 g に収まる
  → 揺れの幅は、学習率とともに小さくなる
- **`lr=0.001`**：300 回では 924 g から 938 g で、まだ下がっている途中
  → 1 歩が小さいぶん、谷底に着くまでの更新回数が増える
- **`n_updates=3000`**：872 g で動かなくなる
- 外れ値に強くなる代わりに、学習率と更新回数を自分で調整することになる

MAE の直線を、MSE の直線と比べる

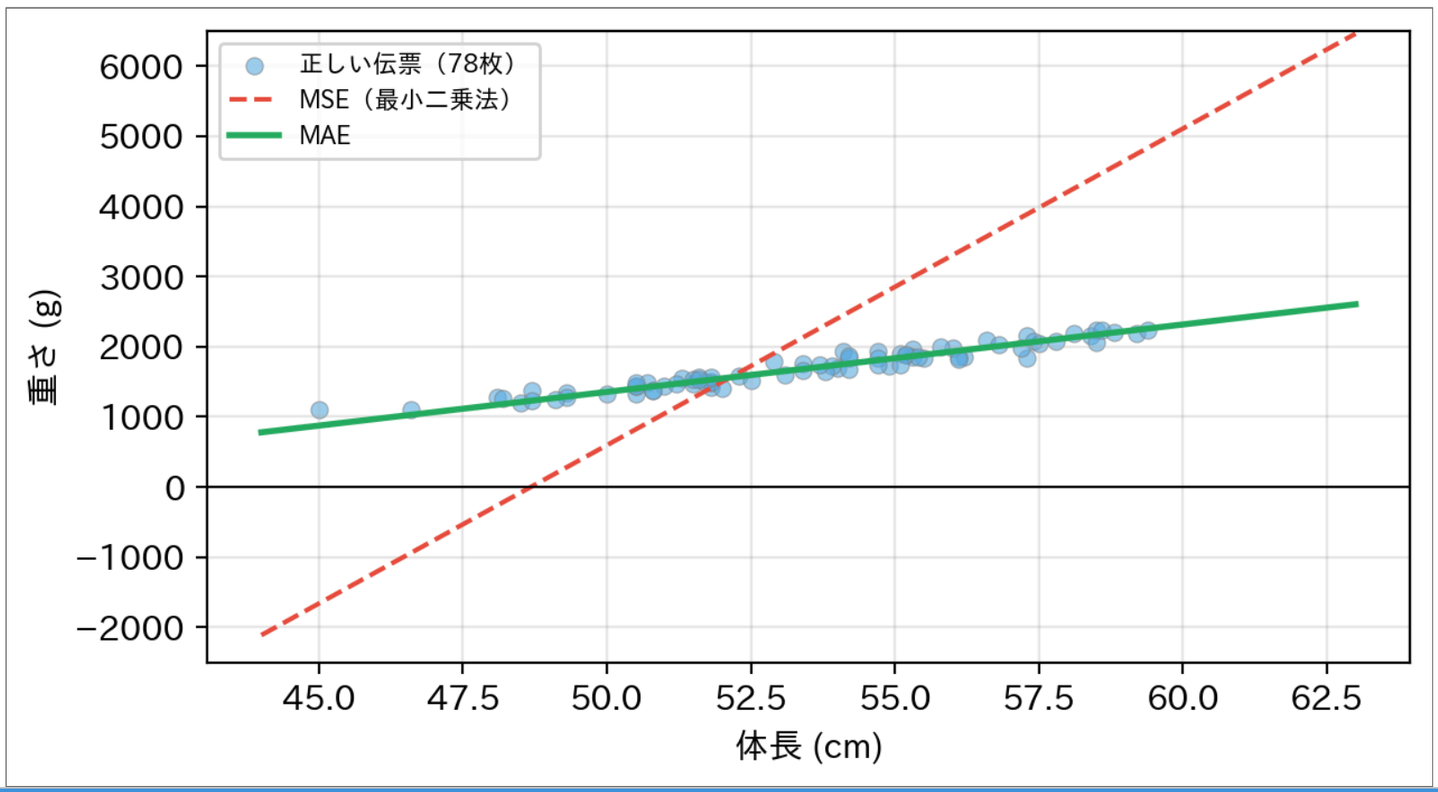
Code

```
a, b, history = fit_mae(lr=0.01, n_updates=1000)
a_mae = (to_gram(a, b, 62) - to_gram(a, b, 45)) / (62 - 45)      # 傾き（g/cm）
print(f"MAE の直線: 体長 1cm あたり {a_mae:.1f} g")
print(f"45 cm の予測: {to_gram(a, b, 45):,.0f} g")
print(f"62 cm の予測: {to_gram(a, b, 62):,.0f} g")
```

Result

```
MAE の直線: 体長 1cm あたり 96.1 g
45 cm の予測: 875 g
62 cm の予測: 2,508 g
```

▸ 描画コード



ⓘ 結果

- 傾きは 451 g/cm から 96 g/cm になった（正しい 80 枚の直線は 91 g/cm）
- 打ち間違いの 2 枚は取り除いていない
  → 2 枚とも、ほかの伝票と同じ 1 枚分の力でしか直線を引けない

14

RANSAC

15

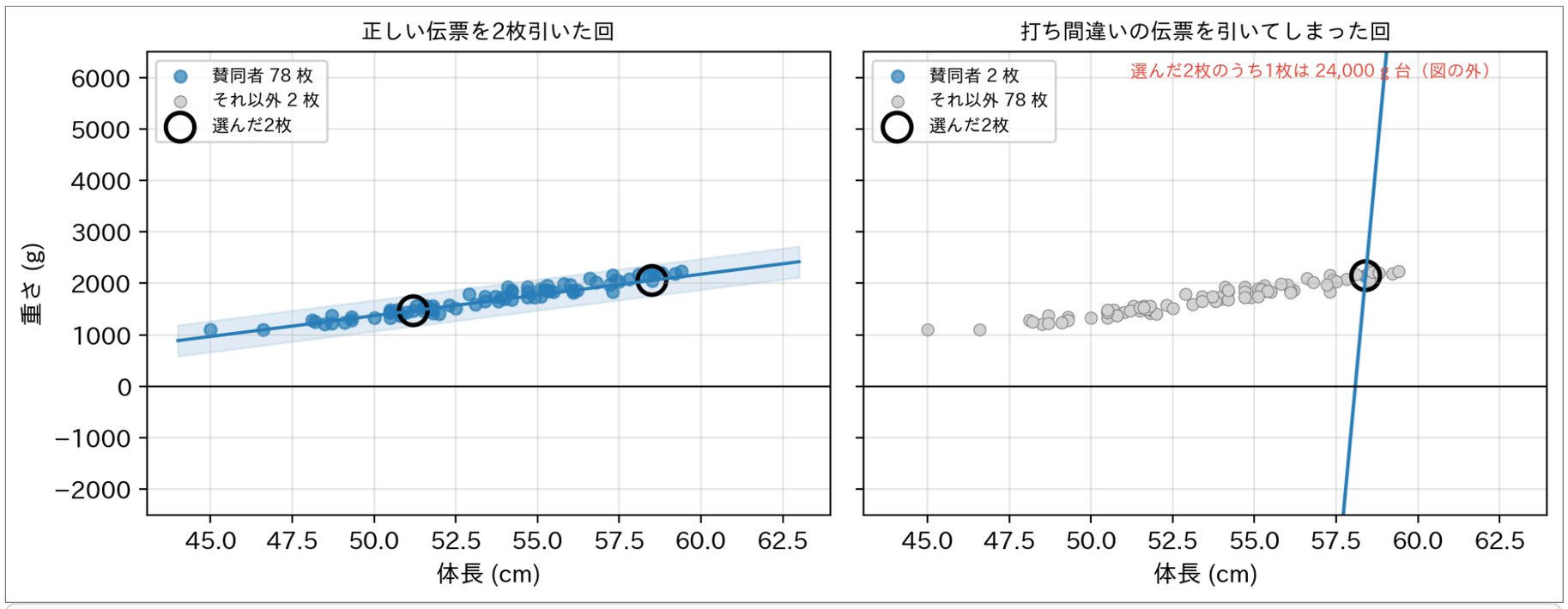
もう1つの対策：賛同者が最多の直線を探す（RANSAC）

- 全部の伝票に合わせるのをやめ、**多くの伝票が賛同する直線**を探す
- 直線は 2 点で決まるので、2 枚の伝票から直線を引き、その直線の近くにある伝票を数える
- 手順：
  1. 伝票から 2 枚をランダムに選ぶ
  2. その 2 点を通る直線を引く
  3. 直線から 300 g 以内にある伝票（**賛同者**、インライア、**inlier**）を数える
  4. 1〜3 を 200 回繰り返し、賛同者が最多だった直線を残す
  5. その賛同者だけで最小二乗法をやり直す
- この手法を **RANSAC（Random Sample Consensus / ランダムに選んだ標本の合意）** と呼ぶ

300 g の根拠： **正しい伝票のズレは最大 206 g だった**。それより少し広くとる

16

1回の試行を図で見る



帯の幅が ±300 g。賛同者の数がその直線の点数。外れ値を引いた回は点数が低く、自然に捨てられる

RANSAC を実装する

Code

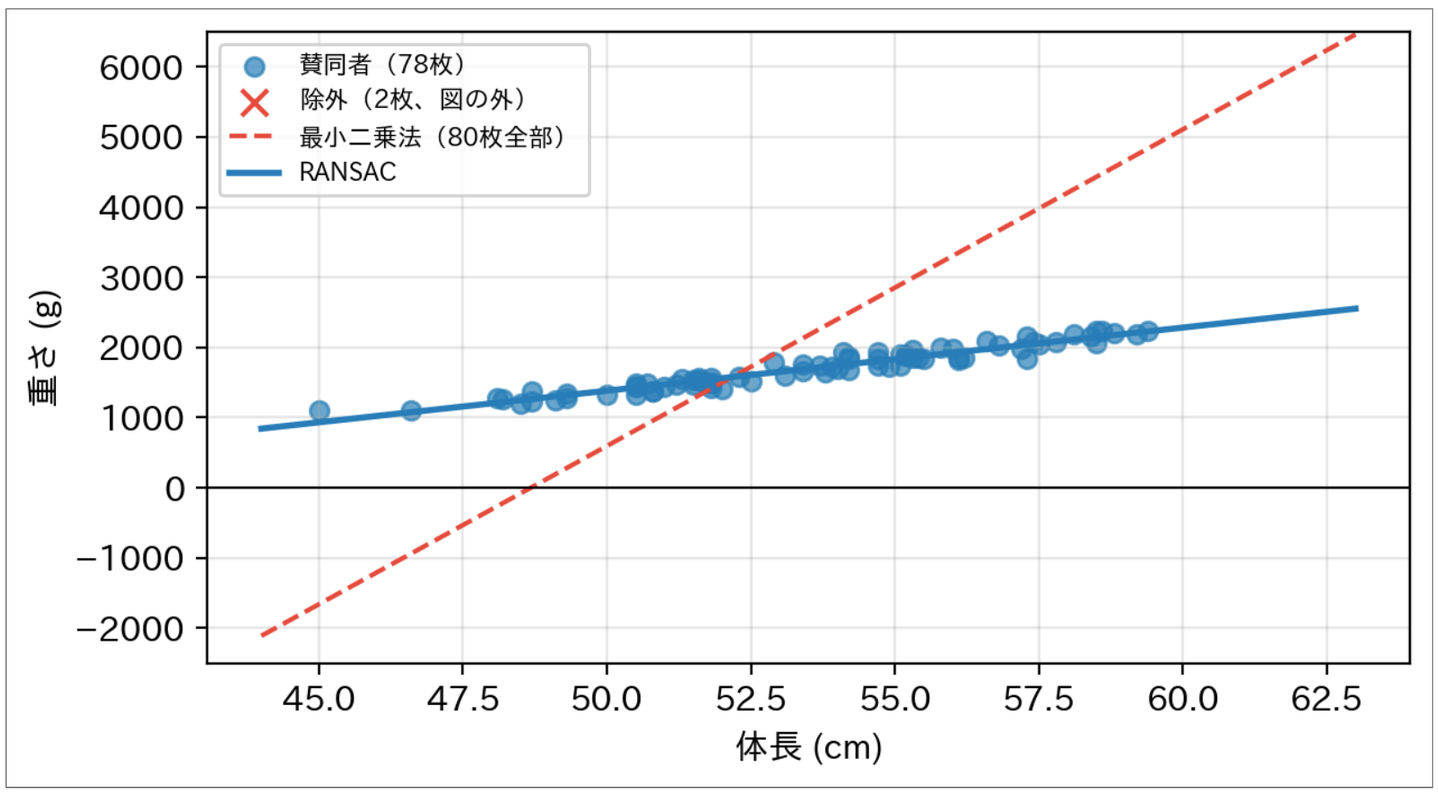
```python
def ransac_line(x, t, threshold, n_trials=200):
    best_count = 0
    for trial in range(n_trials):
        i, j = np.random.choice(len(x), 2, replace=False)      # 手順1：2枚をランダムに選ぶ
        if x[i] == x[j]:                                        # 体長が同じ2枚では直線が引けないので、この回は飛ばす
            continue
        a = (t[j] - t[i]) / (x[j] - x[i])                       # 手順2：2点を通る直線 t = a x + b
        b = t[i] - a * x[i]
        inliers = np.abs(a * x + b - t) < threshold             # 手順3：直線から threshold 以内なら True（賛同者）
        if inliers.sum() > best_count:                          # 手順4：賛同者が最多の直線を覚えておく
            best_count, best_inliers = inliers.sum(), inliers
    reg = LinearRegression().fit(x[best_inliers].reshape(-1, 1), t[best_inliers])   # 手順5：賛同者だけで fit し
    return reg.coef_[0], reg.intercept_, best_inliers

a_r, b_r, inliers = ransac_line(lengths, weights_typo, threshold=300)
print(f"賛同者 {inliers.sum()} 枚、除外 {(~inliers).sum()} 枚")
print(f"RANSAC の直線: 体長 1cm あたり {a_r:.1f} g")
```

Result

```
賛同者 78 枚、除外 2 枚
RANSAC の直線: 体長 1cm あたり 90.0 g
```

▸ 描画コード



ⓘ 結果

- 除外された 2 枚は、ちょうど打ち間違いの 2 枚
- RANSAC の直線は、正しい 78 枚だけで fit した直線（90.0 g/cm）と一致する
- 最小二乗法は全部の伝票への妥協点を探すが、RANSAC は多数派に賛同されない伝票を無視する

（参考）何回試せば十分か

- 2 枚とも正しい伝票を引く確率は w^2（w は正しい伝票の割合）
  → 今回は $w = 78/80$ なので $w^2 \approx 0.95$
- N 回すべて外す確率は $(1 - w^2)^N$
  → $N = 3$ で 0.01 % なので、正しい 2 枚を引くだけなら数回で足りる
- ただし正しい 2 枚を引いても、その 2 枚のばらつきで直線が少し傾き、賛同者を取りこぼすことがある
  → 回数は多めにとる（今回は 200 回）

Tweak しきい値と回数を変える



- **さっきのコード** の引数を **書き換えて実行**（描画コードまで続けて実行すると図で確認できる）
- **bad** を変える行は、伝票のセルから実行し直す

1	**基準を厳しくせよ**	**`threshold=300`** → **`threshold=50`**
2	**打ち間違いを端から真ん中へ移せ**	**`bad = [3, 6]`** → **`bad = [54, 39]`**（体長 53.7 cm と 53.8 cm の魚）
3	**3回だけで決めよ**	**`threshold=300)`** → **`threshold=300, n_trials=3)`**。何度か実行せよ
4	**打ち間違いを20枚にせよ**	**`bad = [3, 6]`** → **`bad = list(range(0, 80, 4))`**

チェックポイント

- **`50`**：賛同者が 37〜39 枚に減り、傾きは実行ごとに 86〜100 g/cm とぶれる
  - → 正しい伝票の半分を外れ値扱いしている
  - → 基準は正しい記録のばらつき（最大 206 g）より広くとる
- 真ん中の 2 枚：最小二乗法の傾きは 92 g/cm とほぼ変わらず、直線全体が 380 g ほど持ち上がる
  - → 端の外れ値は傾きを、真ん中の外れ値は高さを動かす（てこの原理）
  - → RANSAC はどちらでも 2 枚を除外する
- **`n_trials=3`**：4回に1回は賛同者を取りこぼし、傾きが大きくぶれる（−86〜113 g/cm）
  - → 10 回なら 99 %、200 回ならほぼ毎回 78 枚そろう
- 打ち間違い 20 枚：最小二乗法の傾きは 398 g/cm、RANSAC は 92 g/cm
  - → 外れ値が多数派にならない限り、RANSAC は壊れない

（参考）scikit-learn では

- 今日書いた **ransac_line** は、ライブラリでは **RANSACRegressor** に入っている

Code

```
from sklearn.linear_model import RANSACRegressor

ransac = RANSACRegressor(LinearRegression(), residual_threshold=300, random_state=0)
ransac.fit(lengths.reshape(-1, 1), weights_typo)

print(f"賛同者 {ransac.inlier_mask_.sum()} 枚")
print(f"体長 1cm あたり {ransac.estimator_.coef_[0]:.1f} g")
```

Result

```
賛同者 78 枚
体長 1cm あたり 90.0 g
```

今日書いたもの	scikit-learn
ransac_line(x, t, threshold=300)	**RANSACRegressor(LinearRegression(), residual_threshold=300).fit(X, t)**
inliers	**ransac.inlier_mask_**
a_r, b_r	**ransac.estimator_.coef_[0], ransac.estimator_.intercept_**

> ⓘ **（参考）RANSAC の出番**
>
> 直線に限らず、平面・円・2枚の写真の対応関係など「少数の点で決まるモデル」なら同じ手順で使える。画像処理では、写真どうしの対応点から変換を推定するとき（パノラマ合成など）の定番

MAE と RANSAC を比べる

	MSE（最小二乗法）	MAE	RANSAC
外れ値の扱い	二乗して数える	1 枚 1 票で数える	除外する
傾き（g/cm）	451	96	90
外れ値の伝票を特定するか	しない	しない	する（除外された 2 枚）
自分で決めるもの	学習率	学習率と更新回数	しきい値と試行回数

正しい 80 枚で fit した直線の傾きは 91 g/cm

まとめ

- **外れ値**：他の記録から大きく離れた記録
- **MSE** は二乗で測るので、少数の外れ値に直線が引っ張られる
- **MAE**：ズレを絶対値で測る
  - → 勾配に残るのはズレの符号だけで、外れ値も 1 枚 1 票
  - → 1 歩が谷底の近くでも縮まないので、学習率を小さくして揺れを抑える
- **RANSAC**：ランダムな 2 点の直線を何度も引き、賛同者が最多の直線を採る
  - → 外れ値の伝票を除外して fit し直す
- 損失を変えると、fit の結果も、勾配降下法のふるまいも変わる

練習問題：RANSAC を手で1周回す

5 点：$A(0,0)$、$B(1,1)$、$C(2,2)$、$D(3,3)$、$E(2,8)$。しきい値は 0.5。1〜3 は**手計算**、4 と 5 は**Pythonコード**で解け。

1. A と B を通る直線を求め、直線からのズレが 0.5 未満の点（賛同者）を数えよ
2. A と E を通る直線で同じことをせよ
3. RANSAC が最終的に選ぶ直線と、その賛同者を答えよ
4. Python：5 点に MSE の勾配降下法で直線を fit し（標準化なし、学習率 0.1、100 回）、本編の **`ransac_line`** の直線と比較せよ（傾きと切片を表示）
5. Python：4 の勾配を MAE の勾配に書き換えて fit せよ（学習率 0.001、10000 回、傾きと切片を表示）

練習問題の解答例

- A、B を通る直線は $t = x$
  → C、D のズレは 0、E のズレは $|8 - 2| = 6$
  → 賛同者は A, B, C, D の 4 点
- A、E を通る直線は $t = 4x$
  → B のズレは $|1 - 4| = 3$、C は $|2 - 8| = 6$、D は $|3 - 12| = 9$
  → 賛同者は A, E の 2 点
- 賛同者が最多（4 点）なのは $t = x$
  → 賛同者 A, B, C, D だけで最小二乗法をやり直しても、4 点が一直線上なので $t = x$ のまま
- 勾配降下法で 5 点すべてに fit すると $a \approx 1.46$、$b \approx 0.46$
  → E の 1 点に引っ張られ、傾きが 1.46 倍になった
- MAE の勾配降下法では $a \approx 1.00$、$b \approx 0.00$
  → E を除外していないのに、RANSAC と同じ直線になった

▸ 解答例を見る

Result

```
勾配降下法（5点すべて）: a=1.46, b=0.46
RANSAC: a=1.00, b=0.00, 賛同者=[1 1 1 1 0]
MAE の勾配降下法（5点すべて）: a=1.00, b=0.00
```